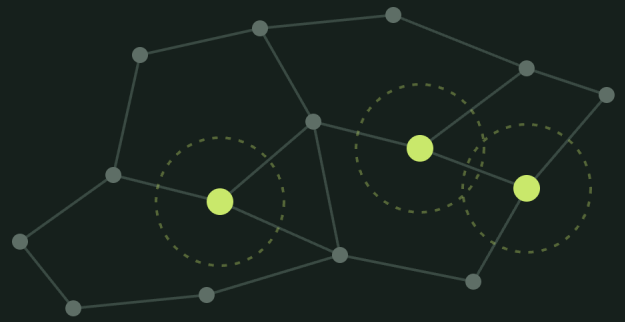
NMIMS MUMBAI · M.SC. DATA SCIENCE



EV charging station optimization for Mumbai

Linear Algebra and Network Science group project: road networks, SVD, Louvain communities and integer programming.

Samiksha Shikare
A021 · Problem & data

Kshiti Vartak
A015 · Pipeline & graph

Rashmi Ashok
A017 · Network analysis

Abin Pillai
A029 · Optimization

The question we set out to answer

If Mumbai builds 12 new EV chargers, where should they go so the most charging demand is within a short drive?

A charger is an expensive, fixed investment that serves its area for years. Mumbai's creeks, highways and narrow peninsula make map distance misleading, so we plan on the real road network.

More coverage

Put as much modeled demand as possible within 2 km of a charger.

Shorter drives

Cut the average driving distance to the nearest charger.

A balanced city

Spread access across regions, not one dense pocket.

Why simple siting rules fail

“Near malls”

Malls may already have chargers, may be poorly connected by road, and serve only one pocket.

“Highest population”

Dense spots sit close together. Their service areas overlap, so extra stations add little.

“Busiest junctions”

Traffic alone ignores whether demand exists nearby or a charger already serves it.

“Straight-line distance”

Drivers follow roads. A creek or highway turns 1 km on a map into 2 km or more.

Data: what is real, what is modeled

Road network

Real

8,574 junctions, 16,284 segments; edges weighted by metres.

Points of interest

Real

47 categories; each POI snapped to its nearest road node.

Existing chargers

Input

The “before” baseline, treated as already-open stations.

Traffic data

Modeled

Simulated for the study; used only in the SVD analysis.

Roads and POIs come from OpenStreetMap. Unsuitable junctions are filtered out, leaving a candidate list far smaller than 8,574 nodes.

Eight modules, run in order

Each Python file does one job. **run_pipeline.py** runs them in order, and every stage passes its output to the next: from raw map data to an interactive map and an analytical report.

01 · network.py
Road graph

02 · demand.py
Demand surface

03 · linalg.py
SVD denoising

04 · centrality.py
Junction importance

05 · communities.py
Communities

06 · optimise.py
Site selection

07 · coverage.py
Coverage metrics

08 · dashboard.py
Map + report

Turning Mumbai into a graph

Junctions become nodes and road segments become edges weighted by length, so every distance in the project is a real driving distance.

IN	OpenStreetMap roads for Mumbai.
DOES	Builds the graph and weights each edge by its length in metres.
OUT	8,574 nodes, 16,284 edges and Dijkstra shortest paths.

```
G = (V, E, w)
A_uv = 1 if (u,v) ∈ E
       else 0

d(u,v) = min over paths P
         Σ w(e), e ∈ P
```

The 2 km coverage test always uses this road distance, never a straight line.

Estimating where demand is

We cannot see real charging sessions, so we use places people go (offices, homes, hospitals, malls) as a proxy for where chargers will be needed.

IN	OSM points of interest in 47 categories.
DOES	Snaps each POI to a node and takes a weighted count.
OUT	A 0–1 demand score per node: the weights w_i.

$$D(v) = \sum_{k=1 \ldots 47} \omega_k \cdot n_k(v)$$

$$\tilde{D}(v) = \frac{D - D_{min}}{D_{max} - D_{min}}$$

$n_k(v)$ is the number of POIs of category k at node v; ω_k is that category's weight.

Cleaning traffic data with SVD

SVD splits a matrix into ranked layers. Large layers are the city's regular traffic patterns; tiny ones are mostly noise, so we keep only the top k.

IN	Traffic matrix T, modeled for the study.
DOES	Decomposes T and keeps the k largest singular values.
OUT	Denoised T_k and the energy kept, E(k).

$$T = U\ \Sigma\ V^T$$

$$T_k = \Sigma\ \sigma_i\ u_i\ v_i^T \quad (i \le k)$$

$$\|T - T_k\| = \sqrt{\Sigma\ \sigma_i^2} \quad (i > k)$$

$$E(k) = \Sigma\ \sigma_i^2\ (i \le k)\ /\ \Sigma\ \sigma_i^2$$

Eckart–Young: T_k is the best rank-k fit. σ_i^2 are the eigenvalues of T^TT.

Ranking junctions by importance

Busy is not enough. Four measures describe how important a junction is, and a weighted score blends them with demand.

Measure	It asks
Degree	Roads meeting here
Betweenness	On many shortest routes?
Closeness	Near everything else?
Eigenvector	Next to important nodes?

$$S(v) = \alpha_1 \cdot \text{demand} + \alpha_2 \cdot \text{degree} + \alpha_3 \cdot \text{betweenness} + \alpha_4 \cdot \text{closeness} + \alpha_5 \cdot \text{eigenvector}$$

$$\Sigma\ \alpha = 1, \quad \alpha \geq 0$$

Eigenvector centrality solves $Ax = \lambda x$ by power iteration. $S(v)$ shortlists the candidates.

Finding natural city regions

Louvain groups junctions that are more tightly linked to each other than to the rest of the city. We use these groups to check fairness.

IN	The road graph from network.py.
DOES	Moves nodes to raise modularity Q, then merges groups.
OUT	About 43 communities for the equity check.

$$Q = (1/2m) \sum_{ij} [A_{ij} - k_i k_j / 2m] \cdot \delta(c_i, c_j)$$

A high Q means far more links inside groups than a random graph with the same degrees would give.

Choosing the 12 sites

A Maximum Coverage Location Problem: pick 12 sites so the most demand has a station within 2 km by road, without wasting sites on overlap.

MODEL	0–1 integer program; existing chargers count as open.
GREEDY	Adds the site with most new coverage; ≥ 63.2% of best.
EXACT	CBC solver proves the best answer within the model.

$$\max \quad \sum w_i \, y_i$$

$$\text{s.t.} \quad y_i \le \sum x_j, \; j \in N_i$$

$$\sum x_j = p \quad (12)$$

$$x_j, \; y_i \in \{0, 1\}$$

$$N_i = \{ \, j : d(i,j) \le R \, \}$$

$x_j = 1$ if we build at j; $y_i = 1$ if demand node i is covered.

What 12 stations change

coverage.py measures each demand node's road distance to its nearest station, before and after the build, weighted by demand.

DEMAND WITHIN 2 KM

20.7% → **88%**

+67.3 percentage points

AVERAGE DRIVE

6.62 km → **1.36 km**

5.26 km shorter, about 79.5%

Under the model's assumptions. Coverage is of modeled demand, not of Mumbai's population.

Limits, and what comes next

WHAT WE DO NOT CLAIM

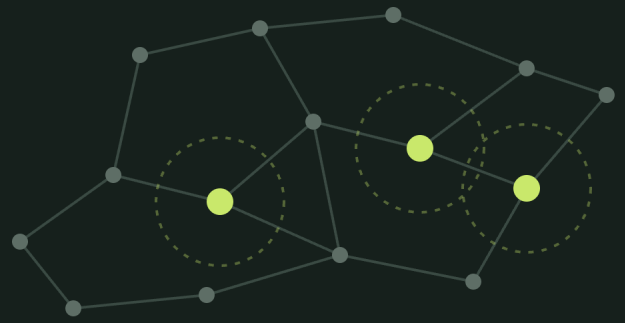
- Traffic inputs are simulated
- Demand is a POI proxy, not EV data
- 88% is modeled demand, not people
- “Optimal” only within the model
- Land, grid and cost not modeled

NEXT STEPS

- Measured traffic and mobility
- Calibrate with EV registrations
- Add capacity, cost, grid limits
- Test sensitivity to R and p
- Extend to other cities

A decision-support tool: real road data, transparent maths, and one repeatable pipeline.

QUESTIONS · DEMO · VIVA



Thank you

Live demo: map.html · report.html

Samiksha Shikare
A021 · Problem & data

Kshiti Vartak
A015 · Pipeline & graph

Rashmi Ashok
A017 · Network analysis

Abin Pillai
A029 · Optimization